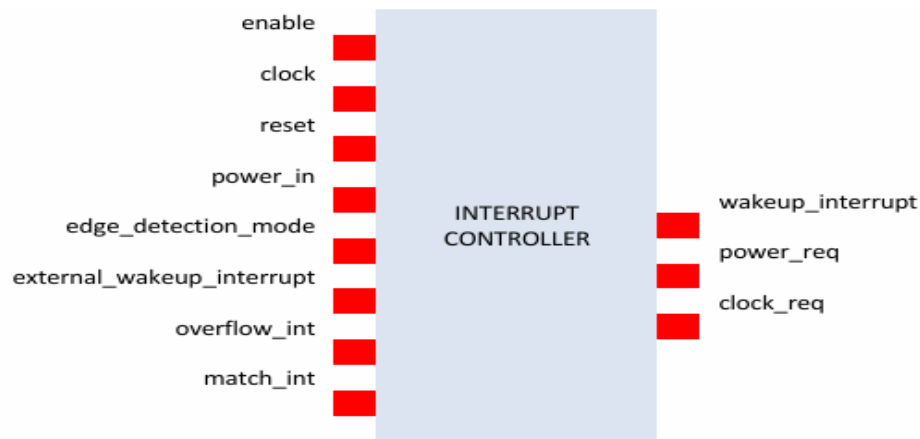


Project Type: SystemVerilog RTL Design and Verification Project

Author: Narasimhamurthy B

INTERRUPT CONTROLLER WITH SELF-CHECKING VERIFICATION ENVIRONMENT

1. Overview



Block Diagram

The Interrupt_controller is a configurable interrupt aggregation and wakeup controller.

The design combines multiple interrupt sources into a single interrupt signal and generates a wakeup interrupt based on programmable edge detection modes.

A complete SystemVerilog verification environment has been developed to verify the functionality of the design using constrained-random stimulus generation, scoreboard-based checking, and functional coverage collection.

2. Design Features

2.1 Interrupt Aggregation

The controller combines three interrupt sources:

Signal Description

overflow_int Counter overflow interrupt

match_int Comparator match interrupt

external_wakeup_interrupt External wakeup source

Combined Interrupt Logic:

$\text{timer_int} = \text{overflow_int} \mid \text{match_int} \mid \text{external_wakeup_interrupt};$

This creates a unified interrupt signal.

2.2 Edge Detection Modes

The controller supports programmable edge detection.

Mode	Value	Function
Disabled	00	No wakeup generation
Rising Edge	01	Detect $0 \rightarrow 1$ transition
Falling Edge	10	Detect $1 \rightarrow 0$ transition
Both Edge	11	Detect any transition

2.3 Power Control Logic

Power request generation:

$\text{power_req} = \text{enable};$

Clock request generation:

$\text{clk_req} = \text{power_in} \ \& \ \text{enable};$

2.4 Reset Behavior

When any of the following occurs:

reset asserted
enable = 0
power_in = 0

the controller:

Clears wakeup_interrupt
Clears timer_int_d

ensuring safe startup operation.

2.5 Edge Detection Mechanism

Previous interrupt state is stored in:

timer_int_d

Wakeup interrupt is generated by comparing:

Current interrupt state
Previous interrupt state

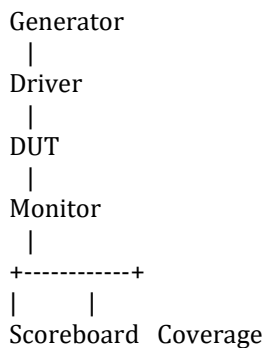
This provides:

- Glitch-free detection
- Deterministic operation
- Hardware-safe implementation

3. Verification Environment

A layered SystemVerilog testbench has been developed.

Architecture:



4. Verification Components

4.1 Generator

Responsibilities:

- Creates transaction objects
- Applies constrained randomization
- Generates 100 random test scenarios

Constraints:

```
enable == 1
power_in == 1
edge_detection_mode inside {[0:3]}
```

4.2 Driver

Responsibilities:

- Receives transactions from generator
- Drives DUT inputs
- Synchronizes stimulus using clock

Communication:

```
mailbox gen2drv
```

4.3 Monitor

Responsibilities:

- Samples DUT inputs and outputs
- Captures actual DUT behavior
- Sends transactions to scoreboard and coverage

Communication:

```
mailbox mon2scb
mailbox mon2cov
```

4.4 Scoreboard

Responsibilities:

- Implements reference model
- Calculates expected wakeup_interrupt
- Compares expected vs actual values

Checks:

Rising Edge Mode
Falling Edge Mode
Both Edge Mode
Disabled Mode

Result:

PASS / FAIL reporting

4.5 Functional Coverage

Coverage collected on:

Edge Detection Modes

00 Disabled
01 Rising
10 Falling
11 Both

Interrupt Sources

overflow_int
match_int
external_wakeup_interrupt

Wakeup Interrupt

0
1

Cross Coverage

Interrupt combinations
Mode vs Wakeup interrupt

5. Verification Results

Simulation Results:

100 Random Transactions
Self-Checking Scoreboard
No Functional Mismatches

Functional Coverage:

98.21% - 100%
(depending on random seed)

All generated test cases successfully passed.

6. Signal Description

Signal	Direction	Description
clk	Input	System Clock
rst	Input	Asynchronous Reset
enable	Input	Global Enable
power_in	Input	Power Availability
edge_detection_mode	Input	Edge Selection
overflow_int	Input	Overflow Interrupt
match_int	Input	Match Interrupt
external_wakeup_interrupt	Input	External Interrupt
wakeup_interrupt	Output	Wakeup Interrupt
power_req	Output	Power Request
clk_req	Output	Clock Request

7. Conclusion

The interrupt controller RTL was successfully verified using a constrained-random self-checking SystemVerilog testbench. The verification environment includes stimulus generation, driver, monitor, scoreboard, and functional coverage collection. All functional requirements were validated, and the design achieved near-complete coverage with zero scoreboard mismatches.